

Files and Exception Handling

Mao

Outline



- **ifstream, ofstream, and fstream**
- **Functions: getline, get, and put**
- **fstream and File Open Modes**
- **Testing Stream States**
- **Exceptions**
- **STL**

ifstream, ofstream, and fstream

C++ provides the ifstream, ofstream, and fstream classes for processing and manipulating files. These classes are defined in the `<fstream>` header file. The `ifstream` class is for reading data from a file, the `ofstream` class is for writing data to a file, and the `fstream` class can be used for reading and writing data in a file.

C++ uses the term stream to describe a flow of data.

`TextFileInput.cpp`

`TextFileOutput.cpp`

- Writing Data to a File
- Reading Data from a File
- Testing End of File

The following code read all data and add the total, and **the last number will be added twice if there are blank characters after the last number.**

```
ifstream input("src/Files_and_Exception_Handling/Files/EOF.txt");
double sum = 0;
double number;
while (!input.eof()) // Continue if not end of file
{
    input >> number; // Read data
    cout << number << " "; // Display data
    sum += number;
}
```

9	5	.	5		6		7	0	.	2		1	.	5	5	\n
1	2		3	.	3		1	2	.	9		8	5	.	6	

Blank character

The reason for this is that when the last number 85.6 is read, the file system does not know it is the last number because there are blank characters after the last number. So, the eof() function returns false. **When the program reads the number again, the eof() function returns true, but the variable number is not changed, because nothing is read from the file.**

There are two ways to fix this problem:

```
while (!input.eof()) // Continue if not end of file
{
    input >> number; // Read data
    if (input.eof()) break;
    cout << number << " "; // Display data
    sum += number;
}
```

```
while (input >> number) // Continue to read
data until it fails
{
    cout << number << " "; // Display data
    sum += number;
}
```

This solution will not work if there is no space at the end.

TestEndOfFile.cpp

ifstream, ofstream, and fstream



- Letting the User Enter a File name

[CheckFile.cpp](#)

- Formatting Output

[WriteFormattedData.cpp](#)

Functions: getline, get, and put

ReadCity.cpp

The `getline` function can be used to read a string that includes whitespace characters and the `get/put` function can be used to read and write a single character.

The `getline` function is:

```
getline(ifstream& input, string s, char delimiterChar)
```

The function stops reading characters when the delimiter character or end-of-file mark is encountered. **If the delimiter is encountered, it is read but not stored in the array.** The third argument `delimiterChar` has a **default value ('\\n')**. The `getline` function is defined in the `iostream` header file.

Functions: getline, get, and put

CopyFile.cpp

The **get** function has two versions:

```
char get() // Return a char
```

```
ifstream* get(char& ch) // Read a character to ch
```

The first version returns a character from the input. The second version passes a character reference argument, reads a character from the input, and stores it in `ch`. This function also returns the reference to the input object being used.

The header for the **put** function is

```
void put(char ch)
```

It writes the specified character to the output object.

Functions: peek and more



```
int peek();
```

The peek function returns the next character in the input sequence, without extracting it: The character is left as the next character to be extracted from the stream.

More ifstream functions can be referred to:

<https://cplusplus.com/reference/fstream/ifstream/>

fstream and File Open Modes

AppendFile.cpp

To open an `fstream` file, specify a file open mode to tell C++ how the file will be used. The file modes are listed in the table below.

<i>Mode</i>	<i>Description</i>
<code>ios::in</code>	Opens a file for input.
<code>ios::out</code>	Opens a file for output.
<code>ios::app</code>	Appends all output to the end of the file.
<code>ios::ate</code>	Opens a file for output. If the file already exists, move to the end of the file. Data can be written anywhere in the file.
<code>ios::trunc</code>	Discards the file's contents if the file already exists. (This is the default action for <code>ios::out</code> .)
<code>ios::binary</code>	Opens a file for binary input and output.

Testing Stream States

The functions `eof()`, `fail()`, `good()`, and `bad()` can be used to test the states of stream operations.

<i>Bit</i>	<i>Description</i>
<code>ios::eofbit</code>	Set when the end of an input stream is reached.
<code>ios::failbit</code>	Set when an operation has failed.
<code>ios::hardfail</code>	Set when an unrecoverable error has occurred.
<code>ios::badbit</code>	Set when an invalid operation has been attempted.
<code>ios::goodbit</code>	Set if none of the preceding bits is set.

These bit values (0 or 1) indicate the state of a stream.

<i>Function</i>	<i>Description</i>
<code>eof()</code>	Returns <code>true</code> if the <code>eofbit</code> flag is set.
<code>fail()</code>	Returns <code>true</code> if the <code>failbit</code> or <code>hardfail</code> flag is set.
<code>bad()</code>	Returns <code>true</code> if the <code>badbit</code> is set.
<code>good()</code>	Returns <code>true</code> if the <code>goodbit</code> is set.
<code>clear()</code>	Clears all flags.

[ShowStreamState.cpp](#)

Exceptions



An exception indicates **an unusual situation** that occurs during a program's execution. Exception handling enables a program to deal with **exceptional situations and continue its normal execution.**

An exception is thrown using **a throw statement** and caught in **a try-catch block.**

Exceptions

A template for a try-throw-catch block may look like this:

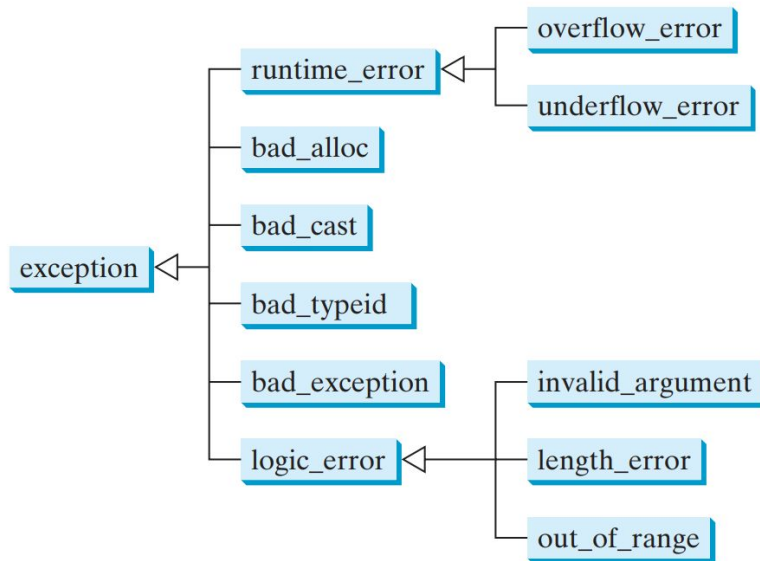
```
try
{
    Code to try;
    Throw an exception with a throw statement or from function if
    necessary;
    More code to try;
}
catch (type ex)
{
    Code to process the exception;
}
```

[TestQuotientWithException.cpp](#)

[TestQuotientWithFunction.cpp](#)

Exceptions

C++ standard exception classes can be used to create exception objects and throw exceptions. These classes are as follows:



- The root class in this hierarchy is **exception** (defined in header `<exception>`). It contains the **virtual function `what()`** that returns an exception object's error.

Exceptions

- The `runtime_error` class (defined in header `<stdexcept>`) is a base class for several standard exception classes that describes runtime errors.
- The `logic_error` class (defined in header `<stdexcept>`) is a base class for several standard exception classes that describes logic errors.
- Classes `bad_alloc`, `bad_cast`, `bad_typeid`, and `bad_exception` describe the exceptions thrown by C++ operators.

[TestQuotientThrowRuntimeError.cpp](#)

[TestBadAllocExceptionDemo.cpp](#)

STL

C++ is an object-oriented language, but recent extensions to the language bring C++ to a higher level. The most significant addition to the language is **the Standard Template Library (STL)**, developed primarily by Alexander Stepanov and Meng Lee.

The library includes three types of generic entities: **containers, iterators, and algorithms.**

Containers includes: **deque, list, map, multimap, set, multiset, stack, queue, priority_queue, and vector.**

Reference



- **Revel for Introduction to C++ Programming and Data Structures by Daniel Liang, 5th edition, Pearson, 2022**